WEEK 3 · LESSON 15 OF 24 · HANDOUT

Data Augmentation: More Data for Free

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=eDRck258bnw — open the lesson online to watch inside the course.

🎓 **Go deeper:** MIT 6.S191 (2023): Convolutional Neural Networks — https://www.youtube.com/watch?v=NmLK_WQBxB4

🎓 **Go deeper:** freeCodeCamp: PyTorch for Deep Learning & Machine Learning — full course — https://www.youtube.com/watch?v=V_xro1bcAuA

Your goal in this lesson: Use torchvision transforms to create realistic variations of training images, measure the gain when data is scarce, and choose augmentations that keep the label true.

The idea

A photo of a maize leaf is still a maize leaf if it is turned a little, moved to the left, flipped or taken in dimmer light. **Data augmentation** creates these variations **randomly every time an image is used for training**, so the network never sees exactly the same picture twice. It learns what matters (the disease spots) and ignores what does not (position, angle, lighting).

It is one of the most effective ways to fight overfitting when you have few images — the usual situation in real projects.

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

def make_shapes(n, kinds=("circle", "square", "triangle"), size=32, seed=0):
    # n noisy grey images of random shapes at random places and sizes
    rng = np.random.default_rng(seed)
    yy, xx = np.mgrid[0:size, 0:size]
    images = np.zeros((n, size, size), dtype=np.float32)
    labels = rng.integers(0, len(kinds), n)
    for i, lab in enumerate(labels):
        r = rng.uniform(5, 10)
        cx, cy = rng.uniform(r + 1, size - r - 1, 2)
        dx, dy = xx - cx, yy - cy
        kind = kinds[lab]
        if kind == "circle":
            mask = dx ** 2 + dy ** 2 <= r ** 2
        elif kind == "square":
            mask = (abs(dx) <= r * 0.85) & (abs(dy) <= r * 0.85)
        elif kind == "triangle":
            mask = (dy >= -r) & (dy <= r) & (abs(dx) <= (dy + r) / 2)
        elif kind == "ring":
            mask = (dx ** 2 + dy ** 2 <= r ** 2) & (dx ** 2 + dy ** 2 >= (0.55 * r) ** 2)
        elif kind == "diamond":
            mask = abs(dx) + abs(dy) <= r
        else:  # cross
            mask = ((abs(dx) <= 0.3 * r) & (abs(dy) <= r)) | ((abs(dy) <= 0.3 * r) & (abs(dx)
<= r))
        images[i][mask] = rng.uniform(0.6, 1.0)
        images[i] += rng.normal(0, 0.15, (size, size))
    images = torch.tensor(np.clip(images, 0, 1)).unsqueeze(1)   # (n, 1 channel, 32, 32)
    return images, torch.tensor(labels)

X_all, y_all = make_shapes(3000)
X_train, y_train = X_all[:2400], y_all[:2400]
X_test, y_test = X_all[2400:], y_all[2400:]
names = ["circle", "square", "triangle"]

fig, axes = plt.subplots(1, 8, figsize=(12, 2))
for ax, img, lab in zip(axes, X_train, y_train):
    ax.imshow(img[0], cmap="gray")
    ax.set_title(names[lab])
    ax.axis("off")
plt.show()
print(X_train.shape, torch.bincount(y_train))
```

```
class ShapeCNN(nn.Module):
    def __init__(self, n_classes=3):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 32 ->
16
            nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 16 -> 8
            nn.Flatten())
        self.head = nn.Sequential(nn.Linear(32 * 8 * 8, 64), nn.ReLU(), nn.Linear(64,
n_classes))

    def forward(self, x):
        return self.head(self.features(x))

def train_model(model, X, y, epochs=5, batch=64, lr=1e-3, params=None, augment=None):
    opt = torch.optim.Adam(params if params is not None else model.parameters(), lr=lr)
    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(len(X))
        for start in range(0, len(perm), batch):
            idx = perm[start:start + batch]
            xb = X[idx]
            if augment is not None:
                xb = torch.stack([augment(img) for img in xb])
            opt.zero_grad()
            F.cross_entropy(model(xb), y[idx]).backward()
            opt.step()
    return model

def accuracy(model, X, y):
    model.eval()
    with torch.no_grad():
        return round((model(X).argmax(dim=1) == y).float().mean().item(), 3)
```

torchvision transforms

```
from torchvision.transforms import v2

augment = v2.Compose([
    v2.RandomHorizontalFlip(),
    v2.RandomAffine(degrees=20, translate=(0.15, 0.15)),   # rotate up to 20°, shift up to 15%
])

img = X_train[0]
fig, axes = plt.subplots(1, 6, figsize=(10, 2))
axes[0].imshow(img[0], cmap="gray"); axes[0].set_title("original")
for ax in axes[1:]:
    ax.imshow(augment(img)[0], cmap="gray")
    ax.set_title("augmented")
for ax in axes:
    ax.axis("off")
plt.show()
```

Each call gives a different random version of the same shape.

Does it help? Only 300 training images

```
X_small, y_small = X_train[:300], y_train[:300]

torch.manual_seed(0)
plain = train_model(ShapeCNN(), X_small, y_small, epochs=40, batch=32)
print("without augmentation:", accuracy(plain, X_test, y_test))

torch.manual_seed(0)
augmented = train_model(ShapeCNN(), X_small, y_small, epochs=40, batch=32, augment=augment)
print("with augmentation:   ", accuracy(augmented, X_test, y_test))
```

With only 300 images, augmentation lifts test accuracy from about **0.93** to about **0.95** — about a third of the mistakes disappear, with no new data collected. Training takes longer per epoch, because every batch is transformed on the fly.

Common augmentations

Transform	What it simulates
`RandomHorizontalFlip`	Mirror image
`RandomRotation`, `RandomAffine`	Tilted camera, object in a different place
`RandomResizedCrop`	Zooming in on part of the object
`ColorJitter`	Different light, camera or time of day
`GaussianBlur`	A slightly out-of-focus photo
`RandomErasing`	Something partly covering the object

The golden rule: keep the label true

Augment only in ways that **do not change the answer**:

- Flipping a leaf photo: fine. Flipping a **digit** "6" upside down turns it into a "9": wrong!
- Strong colour changes are dangerous for crop diseases where colour **is** the symptom (yellow vs green).
- Medical X-rays: flipping left and right can matter (the heart is on the left).

Think about the real camera, the real users and the real world, then choose.

Augment training data only

Validation and test images must stay **untouched** — they represent the real images the model will meet. For real photos, all sets still get the same basic preparation (resize, convert to a tensor, normalise):

```
train_tf = v2.Compose([v2.RandomResizedCrop(32, scale=(0.8, 1.0)),
                       v2.RandomHorizontalFlip(),
                       v2.ColorJitter(brightness=0.3)])
test_tf = v2.Identity()      # nothing random for evaluation
print(train_tf(X_test[0]).shape, test_tf(X_test[0]).shape)
```

Key terms

- **Data augmentation** — random, label-preserving changes to training images.
- **Transform** — one image operation in torchvision (flip, rotate, crop, jitter).

- **Compose** — a chain of transforms applied in order.
- **On-the-fly augmentation** — a new random version every time an image is used.

Remember this: Augmentation gives a small dataset many realistic variations for free — as long as every change keeps the label true and test images stay untouched.

Name: Date:

Activity

Task 1

Add `v2.RandomVerticalFlip()`. Is it safe for these shapes? Would it be safe for digits?

Task 2

Try `degrees=90`. Does stronger augmentation help or hurt?

Task 3

Repeat the experiment with 1,000 training images instead of 300. Is the gain smaller or larger?

Task 4

List three augmentations you would use for photos of cassava leaves taken by farmers on phones, and one you would avoid.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is data augmentation?

   A. Creating random, label-preserving variations of training images
   B. Collecting more photos
   C. Adding labels
   D. Deleting bad images

2. When is augmentation most useful?

   A. Only on the test set
   B. When there are no labels
   C. Only for tables
   D. When there are few training images

3. Why is flipping a handwritten '6' upside down a bad augmentation?

   A. Digits have colour
   B. It can turn it into a '9', changing the true label
   C. Flips are slow
   D. It makes the image smaller

4. Which images should be augmented?

   A. All images
   B. Training images only
   C. Validation images only
   D. Test images only

5. Which transform simulates different lighting?

   A. RandomHorizontalFlip
   B. ColorJitter
   C. MaxPool2d
   D. Flatten

6. Why is ColorJitter risky for some crop-disease images?

   A. It removes the leaf
   B. It makes images bigger
   C. It is too slow
   D. Colour can be the symptom itself

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.